



Universidad autónoma de baja california

Ingeniería en computacion

Internet de las cosas

Lab-taller 14: accion temporizada

Maestro: Dr. Aguilar Noriega Leocundo

Erik Garcia Chávez 01275863

26 de mayo del 2026

Instrucciones:

Usando la plataforma ESP-IDF realizar los ajustes necesarios al prototipo actual para lograr la nueva funcionalidad.

La nueva funcionalidad es hacer que el LED se encienda a un determinado momento del día y se apague en otro momento. Esos momentos se define mediante comandos enviados al dispositivo y tiene una resolución en minutos tomando como referencia las 00:00 (hr:min) como inicio. Es decir que las 12:30 pm corresponde a cantidad de minutos transcurridos desde las 00:00 horas que es este caso sería 750 resultado de $12 \cdot (60) + 30$ minutos. Además, la nueva funcionalidad puede ser habilitada o deshabilitada mediante comando.

Tabla 1. Recursos

Valor	Recurso
0000	RESET
0001	LED (L)
0010	ADC (A)
0011	PWM (P)
0100	Digital (D) - pin 0
0101	Digital (D) - pin 1
0110	Digital (D) - pin 2
0111	Digital (D) - pin 3
1000	Habilitar LED Temporizado (H)
1001	Encender LED Temporizado (N)
1010	Apagar LED Temporizado (F)
1011	-
1100	-
1101	-
1110	-
1111	Servidor (S)

A) El dispositivo IoT prototipo

1. El dispositivo IoT prototipo basado en ESP32, deberá soportar la nueva funcionalidad considerando que solo es para el recurso LED con los siguientes comandos.

CAFE<usuario><longitud><operación>**H**<valor>

donde:

<recurso>: Recurso H, en este caso será la temporización exclusiva para LED

<valor>: En el caso de operación de escritura es necesario un va

Comandos:

a. Habilitar/Deshabilitar Activación Temporizada

CAFE:<usuario><longitud><write>**H**<valor>

H: Corresponde al recurso que en este caso será la temporización exclusiva para LED

<valor>: 1 para habilitar y 0 para deshabilitar

Si el comando es reconocido y se logra la acción se retorna: **ACK:**<valor>, de lo contrario se retorna **NACK**

b. Escribir hora de encendido:

CAFE<usuario><longitud><write>**N**<valor>

N: Corresponde a establecer el momento de día a encender el LED

<valor>: Cantidad de **minutos** correspondiente a la **hr:min** que se quiere que el LED se encienda.

Si el comando es reconocido y se logra la acción se retorna: **ACK:**<valor>, de lo contrario se retorna **NACK**

c. Escribir hora de apagado:

CAFE<usuario><longitud><write>**F**<valor>

F: Corresponde a establecer el momento de día a apagar el LED

<valor>: Cantidad de **minutos** correspondiente a la **hr:min** que se quiere que el LED se apague.

Si el comando es reconocido y se logra la acción se retorna: **ACK:**<valor>, de lo contrario se retorna **NACK**

Desarrollo:

Como primero paso lo que se hizo fue el agregar los nuevos recursos a la tabla de recursos, el cual es habilitar la opción del temporizador, el recurso de escribir la hora de encendido y la hora de apagado del led.

```
1
2 //ahora la operacion y este desma
3 typedef enum{
4     action_none = 0x00,
5     read_esp = 0x1,
6     write_esp = 0x2,
7     cancel_resert_esp=0x03, //cand
8     access_esp = 0x4, //login
9     keep_alive = 0x5,
10 }action_t;
11
12 extern action_t action;
13
14 typedef enum{
15     resert_esp = 0x00,
16     led = 0x1,
17     adc=0x2,
18     pwm=0x3,
19     enable_tmp = 0x8, //hbailitar
20     set_tmp = 0x9, //estabecer la
21     down_tmp = 0xA, //establecer a
22     server = 0xf,
23 }resource_t;
24 extern resource_t resource;
```

Es necesario el uso de funciones, tareas y variables globales que me ayudaran a tener un control sobre el proceso de la temporización.

En el archivo **main.c** encontraremos:

Tenemos la estructura **time_led** en esta estrucutura estaré guardando, el tiempo, cuando se reciben los monitos por parte del usuario, los convierte en uS esto para poder realizar comparaciones con los datos que retorna el recurso de timer del esp32.

La idea es tener un temporizador que estará contando hasta la hora la cual el usuario quiere apagar o prender el led, esto para no estar preguntando cada vez a la API la hora, si no que una vez que se complete el tiempo establecido, preguntara la hora si el tiempo que calculo con el timer interno, queda corto, se compensa ese margen de error, recalculando, los datos del tiempo que debe de esperar están en el miembro **time_set_led**, este guarda el tiempo que esperara para prender el led y **time_down_led** guarda el tiempo que esperara para apagar el led. Estas variables son las que se actualizarán en caso de que el tiempo calculado sea menor, para cuando se prenda el tiempo actual, el tiempo que se consulta a la API debe ser igual o mayor a este. Tanto para prender como para apagar.

Se hace de esta forma porque hay un tiempo muerto en cuanto consulta que puede que a la primera obtiene una respuesta o debe de intentar varias veces. Al final la teoría es que siempre el tiempo cuando se obtenga será mayor que el establecido por el usuario.

Los miembros **hora_set_led**, **minuto_set_led**, **hora_down_led** y **minutos_down_led** guarda el formato hh:mm que convertimos de los minutos datos, esto con el propósito que al momento de traer la hora de la API y después de parsear y convertir de igual manera a un número entero de 8 bits, realizar una comparación y verificar que la condición se haya cumplido, el tiempo se haya cumplido para prender o apagar el led.

El miembro **flag_set_led**, **flag_down_led**, son indicadores que me dirán si ya se prendió el led o si ya se apagó el led, para no volver a ingresar en esa condición.

```
typedef struct{
    uint64_t time_set_led;
    uint64_t time_down_led;
    //para prender
    uint8_t hora_set_led;
    uint8_t minuto_set_led;
    //para apagar
    uint8_t hora_down_led;
    uint8_t minutos_down_led;
    uint8_t flag_set_led;
    uint8_t flag_down_led;
}timer_led_t;
```

Variables, tarea y funciones:

Se tiene **flag_enable_tmp**, es una bandera que me indicara que el recurso ha sido habilitado, en el caso de querer leer, escribir en la hora de prender o apagar el led, el sistema mandara un NACK y mostrara por pantalla que primero se debe de habilitar el recurso

Tenemos la tarea **task_count_time** es la tarea principal, es en donde se estará llevando el control del tiempo del timer, y en donde cuando se llegue la condición de prender o apagar el led, lo hará, esta tarea se crea cuando se escribe en la condición de prender el led, mientras tanto no se crea y una vez el led se ha apagado la tarea se elimina y libera los recursos, porque después de esto no tiene más funcionalidad, cuando se vuelve a introducir otra escritura a prender el led, la tarea se vuelve a crear e inicia el proceso de nuevo.

La función **parse_hora(uint32_t minutos, uint8_t set);**, en esta función los minutos que se recibieron se transforman en su formato hh:mm, guardando cada miembro en los miembros correspondientes de la estructura, así se tiene guardado la hora que el usuario desea prender o apagar el led, en el argumento **set**, se manda 1 cuando se escribe a los miembros que representan prender el led o 0 para Apagar el led.

Las funciones y variables **parsetime()**, **reques_form_API()**, **request**, **parse_time_from_API**, funcionan para traer la hora de la API pasearla, quedarnos solo con la hora que nos interesa, y la función **request_from_API()** devuelve un arreglo de 2 elementos que son la hora y los minutos, pero estos aun van en formato de string

Task_created me indica que la tarea que lleva el conteo ha sido creada, esto para que no cree otra instancia de la misma tarea

Time_init_count y **curren_time**, son variables para llevar el conteo del timer, la primera variable toma como referencia, empieza a contar desde que la tarea es creada y la segunda variable tiene el tiempo actual que se usara para comparar si ya se ha llegado al tiempo establecido en los miembros de la estructura.

```
1 static uint8_t flag_enable_tmp = 0;
2
3 void task_count_time(void *params);
4
5 TaskHandle_t count_time;
6 /*
7  * @brief toma los minutos que se le manda a la f
8  * esto lo guarda en la estrucutra en un formato de
9  * realiza comparacion con la hora traída de la API
10
11  * @params minutos -> los minutos que seran tran
12  * @params set flag que me ayuda a identificar s
13  */
14 void parser_hora(uint32_t minutos, uint8_t set);
15
16 char *parse_time(char *JSON);
17
18 char **reques_from_API(void);
19
20 char *request = NULL;
21 char *parse_time_form_API = NULL;
22
23 static uint8_t task_created = 0;
24
25 uint64_t time_init_count;
26 uint64_t current_time;
```

En la tarea **tcp_proccess_task()**, se agregan las opciones de los nuevos recursos, primero revisaremos cuando el usuario intenta leer.

Cuando se intenta leer si el recurso está habilitado, el sistema no mostrara error, ya que se busca conocer si está habilitado o no, por lo que mandamos lo que la bandera contiene en este instante que se realiza la consulta si está habilitado mandaría 1, si no un 0

```
case enable_tmp:{
    send_info.op_type = OP_NACK;
    send_info.format_request.value[0] = flag_enable_tmp; // mandara 0
    send_info.format_request.len = 1;
    ret = send_message();

}break;
```

En el caso que se quiera **leer la hora de encender el led**, lo primero que tiene que hacer es verificar que el recurso se encuentra habilitado en otro caso no podrá leerlo y mandara un **NACK**, en el caso que si este habilitado, como el sistema guarda tanto el tiempo que resta para la hora de prender en uS y la presentación entera de esta hora, se hace una operación simple para convertir la hora de vuelta en minutos que será lo que se enviara, donde su máximo ese 4 bytes, que el sistema puede guardar, da un margen de 49 días por lo que es un rango aceptable.

```
case set_tmp:{
    if(flag_enable_tmp != 1){

        len = snprintf(buffer, sizeof(buffer), "\r\nrecurso temporizador no habilitado\r\n");
        uart_write_bytes(UART_NUM_0, UART_RED, strlen(UART_RED));
        uart_write_bytes(UART_NUM_0, buffer, len);
        uart_write_bytes(UART_NUM_0, UART_RESET, strlen(UART_RESET));
        send_info.op_type = OP_NACK;
        ret = send_message();
    }
    else{

        uint32_t minutos = ( time_led.hora_set_led * 60 ) + time_led.minuto_set_led; // (hora *
        send_info.op_type = OP_ACK;
        memcpy(send_info.format_request.value, &minutos, 4);
        send_info.format_request.len = 4;
        ret = send_message();
    }

}

}break;
```

Para el caso de **down_led**, es el mismo solo que hacemos operaciones con el miembro correspondiente:

```
1 case down_tmp:{
2     if(flag_enable_tmp != 1){
3         len = snprintf(buffer, sizeof(buffer), "\r\nrecurso temporizador no habilitado\r\n");
4         uart_write_bytes(UART_NUM_0, UART_RED, strlen(UART_RED));
5         uart_write_bytes(UART_NUM_0, buffer, len);
6         uart_write_bytes(UART_NUM_0, UART_RESET, strlen(UART_RESET));
7         send_info.op_type = OP_NACK;
8         ret = send_message();
9     }
10    else{
11        uint32_t minutos = ( time_led.hora_down_led * 60 ) + time_led.minutos_down_led; // (hora
12        send_info.op_type = OP_ACK;
13        memcpy(send_info.format_request.value, &minutos, 4);
14        send_info.format_request.len = 4;
15        ret = send_message();
16    }
17
18 }break;
```

Ahora para el proceso de escribir, es más largo, primero el caso de habilitar o deshabilitar el recurso, primero debe de verificar que es lo que el usuario, desea, en el caso de habilitar el recurso, activamos la bandera que nos indica que el servicio está habilitado, regresando un ACK, aun no se inician recursos, solo se le dijo al sistema que este servicio está listo para operar, en el caso de deshabilitar, limpiamos todo lo

relacionado con el temporizador, miembros, eliminados la tarea en el caso que ya se estuviera ejecutando, banderas las limpiamos, para poder volver a empezar sin errores en el caso que se vuelva a habilitar .

```
case enable_tmp :{
    //si en el recurso H llega el valor 1 indica que se quiere habilitar el recurso
    //pero en el caso de que llegue 0 se requiere que se deshabilite el recurso

    if(frame->value[0] != 1 ){
        //indicamos que entonces se quiere deshabilitar el recurso
        //si estaba activo ahora lo apagamos
        flag_enable_tmp = 0;

        //ahora necesitamos apagar el conteo, eliminamos la tarea
        vTaskDelete(count_time);

        //elimino y reinicio todo lo que necesita el sistema del led

        time_led.time_set_led = 0;
        time_led.time_down_led = 0;
        time_led.flag_set_led = 0;
        time_led.flag_down_led = 1;
        time_led.hora_set_led = 0;
        time_led.minuto_set_led = 0;
        time_led.hora_down_led = 0;
        time_led.minutos_down_led = 0;
        task_created = 0;
        time_init_count = 0;

        send_info.op_type = OP_ACK;
        send_info.format_request.len = 1;
        send_info.format_request.value[0] = frame->value[0]; //valor de 0 o 1
        ret = send_message();
    }
    else{
        //en otro caso entonces lo que se solicita es habilitar el recurso
        flag_enable_tmp = 1; //indicamos que ahora el recurso es activo
        send_info.op_type = OP_ACK; //indiamos un ACK se pudo
        send_info.format_request.len = 1;
        send_info.format_request.value[0] = 1;
        ret = send_message();
    }
}
}break;
```

El caso de escribir la hora de prener el led, primero debe de verificar que el recurso esta activo:

```
case set_tmp:{

    if(flag_enable_tmp !=1){

        char msg[80];
        len = sprintf(msg, sizeof(msg), "\r\n no se ha habilitado el recurso -> devolviendo NACK al servidor\n\r");
        uart_write_bytes(UART_NUM_0, UART_RED , strlen(UART_RED));
        uart_write_bytes(UART_NUM_0, msg , len);
        uart_write_bytes(UART_NUM_0, UART_RESET , strlen(UART_RESET));
        send_info.op_type = OP_NACK;
        send_info.format_request.value[0] = 0;
        ret = send_message();
    }
    else{
```

En el caso que si este activo, entonces lo que hará es verificar primero que el frame venga completo, si el frame tiene menos bits de los máximos que debería de tener se indica que hubo un error, el frame está incompleto, después debemos de verificar que los minutos, no sean ni un numero negativo ni que sea la hora 0, entonces se verifica, si no es ninguno de estos casos quiere decir que el numero que se ingreso es un numero positivo mayor que 0 y el frame está completo.


```

uint8_t value_len = (frame->len > 5) ? (frame->len - 5) : 0;

if(value_len > 0){
    int negativo = (frame->value[0] & 0x80) != 0;
    int cero = 1;
    for (int i = 0; i < value_len; i++) {
        if (frame->value[i] != 0) { cero = 0; break; }
    }

    if(negativo){
        len = snprintf(buffer, sizeof(buffer), "\r\n ERROR! --> el numero recibido es negativo, opeacion imposible\r\n");
        uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
        uart_write_bytes(global_uart.NUM_PORT, buffer, len);
        uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
        send_info.op_type = OP_NACK;
        send_info.format_request.value[0] = 0;
        ret = send_message();
    }

    else if(cero){
        len = snprintf(buffer, sizeof(buffer), "\r\n ERROR! --> numero es 0, debe ser un numero > 0\r\n");
        uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
        uart_write_bytes(global_uart.NUM_PORT, buffer, len);
        uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));

        send_info.op_type = OP_NACK;
        send_info.format_request.value[0] = 0;
        ret = send_message();
    }
}

```

Ahora lo que se hace es establecer el tipo de dato correcto, aplicación la función **ntohl** o **ntohs** dependiendo el caso, verificando cuando bytes nos quedan una vez eliminado todos los bytes anteriores a la parte de value del frame, si este supera los 4 bytes indicamos que es un número que no se puede procesar

```

1 len = snprintf(buffer, sizeof(buffer), "\r\n bytes restantes : %u", value_len);
2   uart_write_bytes(UART_NUM_0, UART_GREEN, sizeof(UART_GREEN));
3   uart_write_bytes(UART_NUM_0, buffer, len);
4   uart_write_bytes(UART_NUM_0, UART_RESET, sizeof(UART_RESET));
5
6   uint32_t minutos=0;
7   if(value_len == 2){
8
9       uint16_t tmp = 0;
10      memcpy(&tmp, frame->value, 2);
11      minutos = ntohs(tmp);
12  }
13  else if(value_len == 4){
14
15      memcpy(&minutos, frame->value, 4);
16
17      minutos = ntohl(minutos);
18  }
19  else if(value_len == 1){
20
21      minutos = frame->value[0];
22  }
23  }
24  else{
25
26      len = snprintf(buffer, sizeof(buffer), "\nrtamaño del numero excede, el vaor maximo debe entrar en 32 bits\n\r");
27      uart_write_bytes(UART_NUM_0, UART_RED, strlen(UART_RED));
28      uart_write_bytes(UART_NUM_0, buffer, len);
29      uart_write_bytes(UART_NUM_0, UART_RESET, strlen(UART_RESET));
30      send_info.op_type = OP_NACK;
31      send_info.format_request.value[0] = 0;
32      ret = send_message();
33
34      break;
35  }

```

Para el caso que el numero ingresa entre máximo 4 bytes, entonces lo primero que se hace es mandar a llamar a la hora desde la API, esto para verificar que el tiempo que el usuario introdujo no haya pasado, por ejemplo, si el usuario ingreso las 10:30 AM y en el frame envió que quiere apagarlo a las 9:30 AM no será posible, o si es la hora exacta, igual no será posible, debe de haber un margen de tiempo de al menos 1 minuto.

```

char **tokens = reques_from_API();
uint8_t horas_API = (uint8_t)atoi(tokens[0]);
uint8_t minutos_API = (uint8_t)atoi(tokens[1]);

uint32_t minutos_from_API = (horas_API * 60) + minutos_API;

int32_t minutos_faltantes = minutos - minutos_from_API;

if(minutos_faltantes < 0 ){
    len = snprintf(buffer, sizeof(buffer), "\r\nHora del día ya pasada, intente de nuevo\r\n");
    uart_write_bytes(UART_NUM_0, UART_RED, sizeof(UART_RED));
    uart_write_bytes(UART_NUM_0, buffer, len);
    uart_write_bytes(UART_NUM_0, UART_RESET, sizeof(UART_RESET));

    send_info.op_type = OP_NACK;
    send_info.format_request.value[0] = 0;
    ret = send_message();

    break;
}
else if(minutos_faltantes == 0){
    len = snprintf(buffer, sizeof(buffer), "\r\nhora solicitada coincide con la hora actual, se requ");
    uart_write_bytes(UART_NUM_0, UART_RED, sizeof(UART_RED));
    uart_write_bytes(UART_NUM_0, buffer, len);
    uart_write_bytes(UART_NUM_0, UART_RESET, sizeof(UART_RESET));

    send_info.op_type = OP_NACK;
    send_info.format_request.value[0] = 0;
    ret = send_message();
}

```

En caso de que el tiempo sea dentro de una franja de horario aceptable, se asigna a los miembros correspondientes la cantidad de uS a esperar, se inicia la tarea e iniciamos la referencia del conteo del timer.

```

else if(minutos_faltantes > 0 ){
    uint64_t segundos = (uint64_t)minutos_faltantes * 60;
    time_led.time_set_led = segundos * 1000000ULL;

    parser_hora(minutos,1);

    if(task_created != 1 ){

        xTaskCreate(task_count_time, "count_time", 4096, NULL, 8, &count_time);
        task_created =1;

    }

    time_init_count = 0;
    current_time = 0;

    send_info.op_type = OP_ACK;
    memcpy(send_info.format_request.value, &minutos, value_len);
    send_info.format_request.len = value_len;
    send_info.format_request.value[0] = minutos;
    ret = send_message();

    time_init_count =esp_timer_get_time();

}

}
}
else{
    char msg[80];
    len = snprintf(msg, sizeof(msg), "\r\nERROR! --> no hya suficientes parametros en el frame. \n\r");
    uart_write_bytes(UART_NUM_0, UART_RED , strlen(UART_RED));
    uart_write_bytes(UART_NUM_0, msg , len);
    uart_write_bytes(UART_NUM_0, UART_RESET , strlen(UART_RESET));

    //mandando NACK
    send_info.op_type = OP_NACK;
    send_info.format_request.value[0]=0;
    ret = send_message();
}

k;

```

En el caso de **down_led**, el proceso es exactamente igual, solo que en este caso no está la función que inicia variables o la tarea, tan solo se asigna a los miembros de la estructura que representan la hora de apago el tiempo a esperar, la hora y los minutos.

La tarea **task_count_time()**, esta tarea se mantiene en espera que el timer llegue al periodo de tiempo establecido, una vez que llegue el sistema consulta a la API la hora que tiene, convierte lo que regresa en forma de string a enteros con ayuda de `atoi`, lo convierte en minutos para realizar la comparación si el tiempo actual es menor que el tiempo que se supone que tiene que llegar entonces se realiza un ajuste en donde actualizamos el tiempo a esperar, en los miembros, por lo que vuelve a contar y espera a llegar al tiempo objetivo actualizado, así es hasta que tiempo de API sea mayor que el tiempo objetivo

```

void task_count_time(void *params){

    char **tokens = NULL;
    uint8_t hora_API = 0;
    uint8_t minutos_API = 0;

    while(1){

        current_time = esp_timer_get_time() - time_init_count;

        if(time_led.flag_set_led != 1 && (current_time >= time_led.time_set_led)){
            tokens = reques_from_API();

            hora_API =(uint8_t)atoi(tokens[0]);
            minutos_API =(uint8_t)atoi(tokens[1]);

            uint32_t actual = hora_API * 60 + minutos_API;
            uint32_t objetivo = time_led.hora_set_led * 60 + time_led.minuto_set_led;

            if(actual >= objetivo){

                led_state = 1;
                time_led.flag_set_led =1;
                time_led.flag_down_led = 0;
                tokens = NULL;
                hora_API = 0;
                minutos_API = 0;
                gpio_set_level(OUTPUT_PIN, led_state);

            }
            else{
                uint8_t rest_hour = time_led.hora_set_led - hora_API;
                uint8_t rest_minutos = time_led.minuto_set_led - minutos_API;

                uint32_t minutos_ajuste = (rest_hour * 60) + rest_minutos;
                uint64_t segundos = (uint64_t)minutos_ajuste * 60;
                time_led.time_set_led = segundos * 1000000ULL;

            }
        }
    }
}

```

El proceso para apagar el led, es el mismo, solo que en este cuando se apaga el led, tambien se reinician todas las varibales, miembros de la estrucutra y se elimina la tarea

```

else if(time_led.flag_down_led != 1 && (current_time >= time_led.time_down_led)){

    tokens = reques_from_API();

    hora_API =(uint8_t)atoi(tokens[0]); //convierto las horas de ASCII a horas entros
    minutos_API =(uint8_t)atoi(tokens[1]);

    uint32_t actual = hora_API * 60 + minutos_API;
    uint32_t objetivo = time_led.hora_down_led * 60 + time_led.minutos_down_led;

    if(actual >= objetivo){

        led_state = 0;
        time_led.flag_down_led = 1;
        time_led.flag_set_led = 0;
        tokens = NULL;
        gpio_set_level(OUTPUT_PIN, led_state);
        time_led.time_set_led = 0;
        time_led.time_down_led = 0;
        time_led.flag_set_led = 0;
        time_led.flag_down_led = 1;
        time_led.hora_set_led = 0;
        time_led.minuto_set_led = 0;
        time_led.hora_down_led = 0;
        time_led.minutos_down_led = 0;
        task_created = 0;
        time_init_count = 0;

        vTaskDelete(NULL); //entonces elimino la tarea, el proceso se completo

    }
}

```